SC2001 Project 1



Team members: Dylan, Jiashun, Valli

Introduction

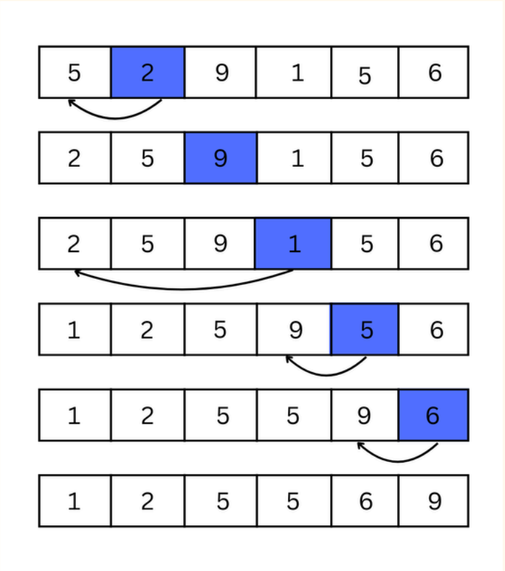
Hybrid Sorting

The hybrid sorting algorithm combines **Merge Sort** and **Insertion Sort**

Purpose: Overcome Merge Sort's inefficiency for small arrays by switching to Insertion Sort when the array size is below the threshold size (S)

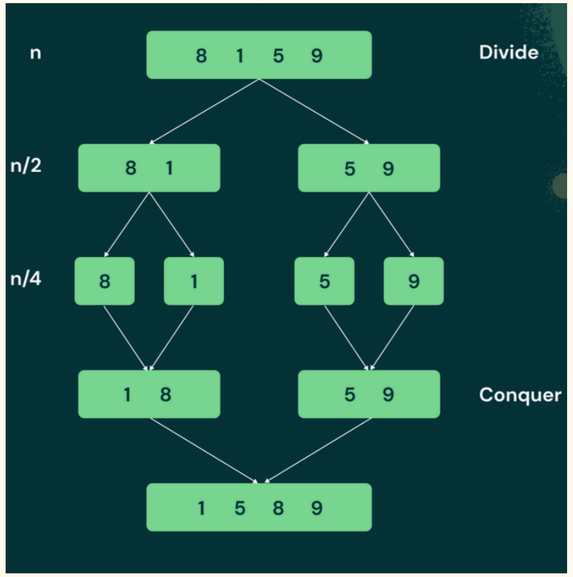
The threshold size (S) determines when to switch from Merge Sort to Insertion Sort

Insertion Sort

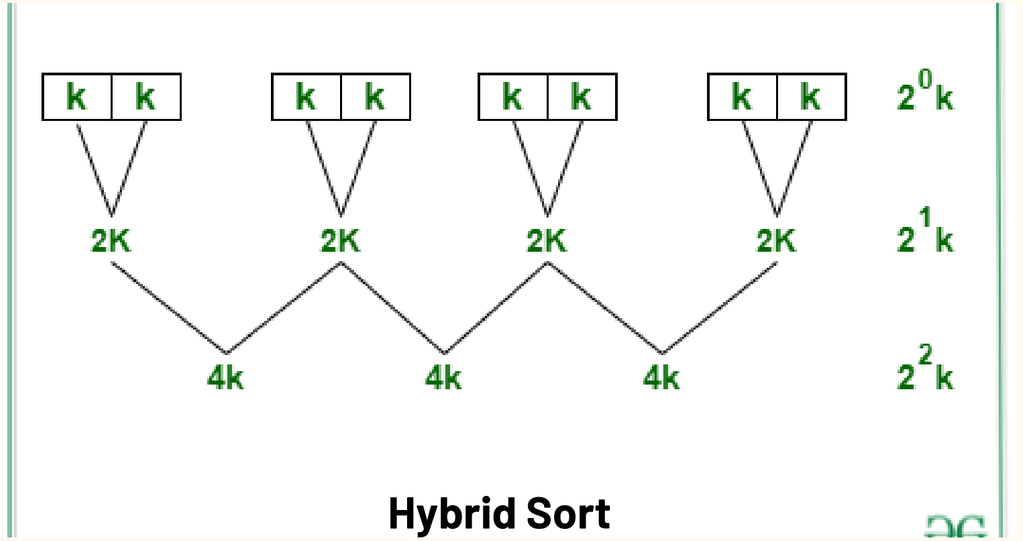


+

Merge Sort



=



Hybrid Sort

Algorithm Implementation

Key Comparisons, Merge Sort & Insertion Sort

1 Key Comparisons

```python
class KeyComparisons:
    def __init__(self):
        self.key_comparisons = 0

    def less_than(self, a, b):
        self.key_comparisons += 1
        return a < b

    def less_than_or_equal(self, a, b):
        self.key_comparisons += 1
        return a <= b

    def reset(self):
        self.key_comparisons = 0

    def get(self):
        return self.key_comparisons
```

2 Merge Sort

```python
def merge(list1, list2, comp_obj: KeyComparisons):
    sorted_list = []
    i = j = 0
    len1, len2 = len(list1), len(list2)

    while i < len1 and j < len2:
        if comp_obj.less_than_or_equal(list1[i], list2[j]):
            sorted_list.append(list1[i])
            i += 1
        else:
            sorted_list.append(list2[j])
            j += 1

    # Append the remaining elements from both lists
    sorted_list.extend(list1[i:])
    sorted_list.extend(list2[j:])
    return sorted_list

def merge_sort(arr, comp_obj: KeyComparisons):
    #arrays with just 0 or 1 elements are already sorted, so just return
    if len(arr) <= 1:
        return arr
    #middle index that splits the array
    mid = len(arr) // 2
    #recursively sort left half of array
    left = merge_sort(arr[:mid], comp_obj)
    #recursively sort right half of array
    right = merge_sort(arr[mid:], comp_obj)
    #merge two sorted halves into single array
    return merge(left, right, comp_obj)
```

3 Insertion Sort

```python
def swap(arr, i, j):
    arr[i], arr[j] = arr[j], arr[i]

def insertion_sort(arr, comp_obj: KeyComparisons):
    #start from second element (index 1) and traverse though each element in array
    for i in range(1, len(arr)):
        #compare current element with all previous elements, moving backwards
        for j in range(i, 0, -1):
            #if the current element is smaller than prev, swap
            if comp_obj.less_than(arr[j], arr[j - 1]):
                swap(arr, j, j - 1)
            #otherwise it is in correct pos so stop checking
            else:
                break
    #return sorted array
    return arr
```

Algorithm Implementation

Hybrid Sort

```
def hybrid_sort(arr, S, comp_obj: key_comparisons.KeyComparisons):
    #if array size <= S, use insertion sort
    if len(arr) <= S:
        return insertion_sort(arr, comp_obj)

    #otherwise split array and sort halves recursively using hybrid sort
    else:
        mid = len(arr) // 2
        left = hybrid_sort(arr[:mid], S, comp_obj)
        right = hybrid_sort(arr[mid:], S, comp_obj)
        #merge sorted halves into one array
        return merge(left, right, comp_obj)
```

Implementation

Dataset Generation

```python
import random

def generate_input_sizes():
    input_sizes = []
    for i in range(10):
        input_sizes.append((i+1) * 1000)
        input_sizes.append((i+1) * 10000)
        input_sizes.append((i+1) * 100000)
        input_sizes.append((i+1) * 1000000)
    input_sizes = sorted(set(input_sizes))
    return input_sizes

def generate_datasets(seed=42):
    random.seed(seed)
    input_sizes = generate_input_sizes()
    input_data = []
    for s in input_sizes:
        data = [random.randint(1, s) for _ in range(s)]
        input_data.append(data)
    return input_data, input_sizes

def verify_datasets(input_data, input_sizes):
    for i in range(len(input_data)):
        print("Array Size:", len(input_data[i]))
        print("Min of this array:", min(input_data[i]))
        print("Max of this array:", max(input_data[i]))
        print()
```

Time complexity

Insertion sort

best case

“what the algorithm likes to ‘eat’ “

when the list is already sorted

for example: [0 1 2 3 4 5 6]

1 comparison per outer loop

O(n)

```
[0 1 2 3 4 5 6]  i=1, 1 comparison
[0 1 2 3 4 5 6]  i=2, 1 comparison
[0 1 2 3 4 5 6]  i=3, 1 comparison
[0 1 2 3 4 5 6]  i=4, 1 comparison
[0 1 2 3 4 5 6]  i=5, 1 comparison
[0 1 2 3 4 5 6]  i=6, 1 comparison
```

worst case

“what the algorithm hates to ‘eat’ “

when the list is reversed

example: [6, 5, 4, 3, 2, 1, 0]

every element at position i must travel to position 0.

1 + 2 + ... + (n-1)

O(n^2)

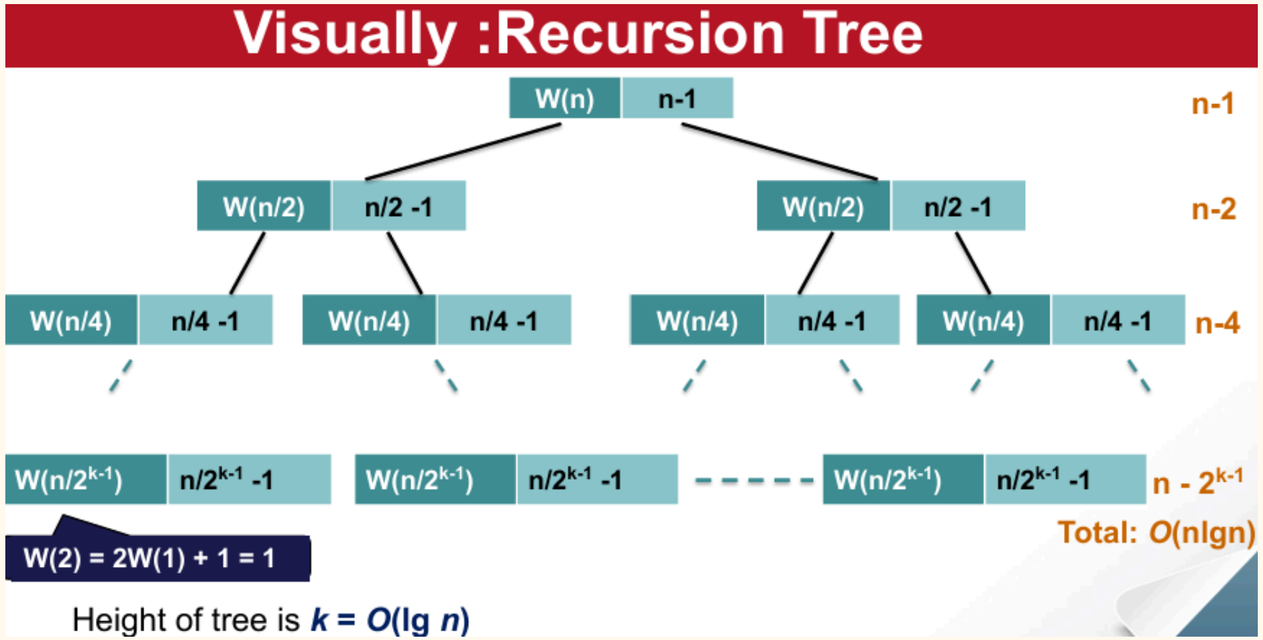
```
[6 5 4 3 2 1 0] i=1, 1 comparisons
[5 6 4 3 2 1 0] i=2, 2 comparisons
[4 5 6 3 2 1 0] i=3, 3 comparisons
[3 4 5 6 2 1 0] i=4, 4 comparisons
[2 3 4 5 6 1 0] i=5, 5 comparisons
[1 2 3 4 5 6 0] i=6, 6 comparisons
```

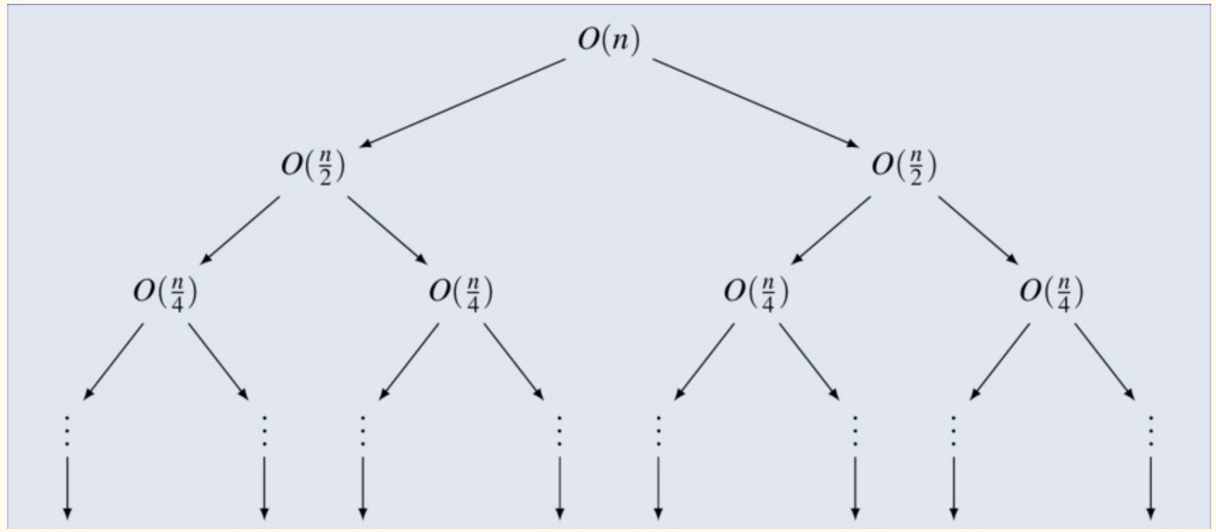
Based on the lecture on insertion sort:

- We can apply it to the Hybrid sort where there’s **(n/s)** sub arrays
- Best case: **(n/s) * s = O(n)**
- Worst case: **(n/s) * (s^2)= O(ns)**

Time complexity

Merge sort





s s s s s s s s s s s s s s s s s s s s s s s s s s s s s s s s

Based on the lecture on merge sort:

- Merge sort will divide the array until it reaches size **s**. No.of sub array = **n/s**
- Height of the tree is **O(lg n/s)** ---> **n/s** = no.of sub arrays at the root nodes
- Best/Worst case: **O(n lg n/s)**

Time complexity

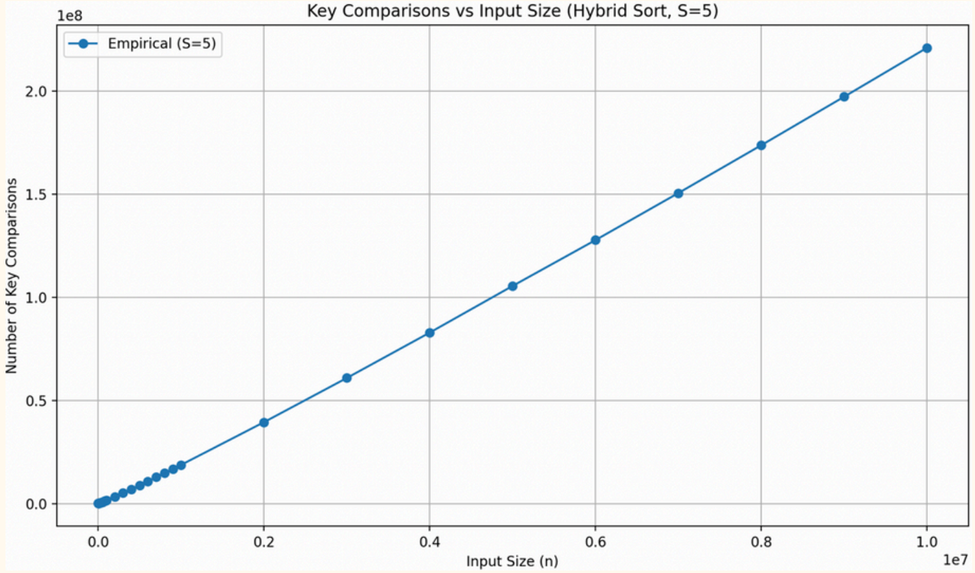
Hybrid sort

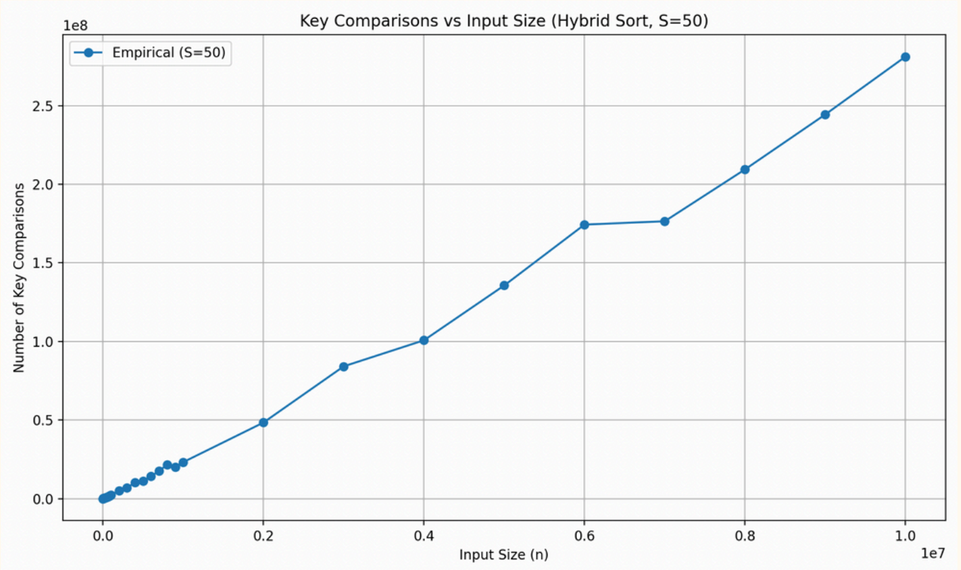
Based on insertion and merge sort:

- Best case: **O(n + n lg n/s)**
- Worst and Average case: **O(ns + n lg n/s)**

Fixed s value

No. of Key comparison Vs Different size (n)

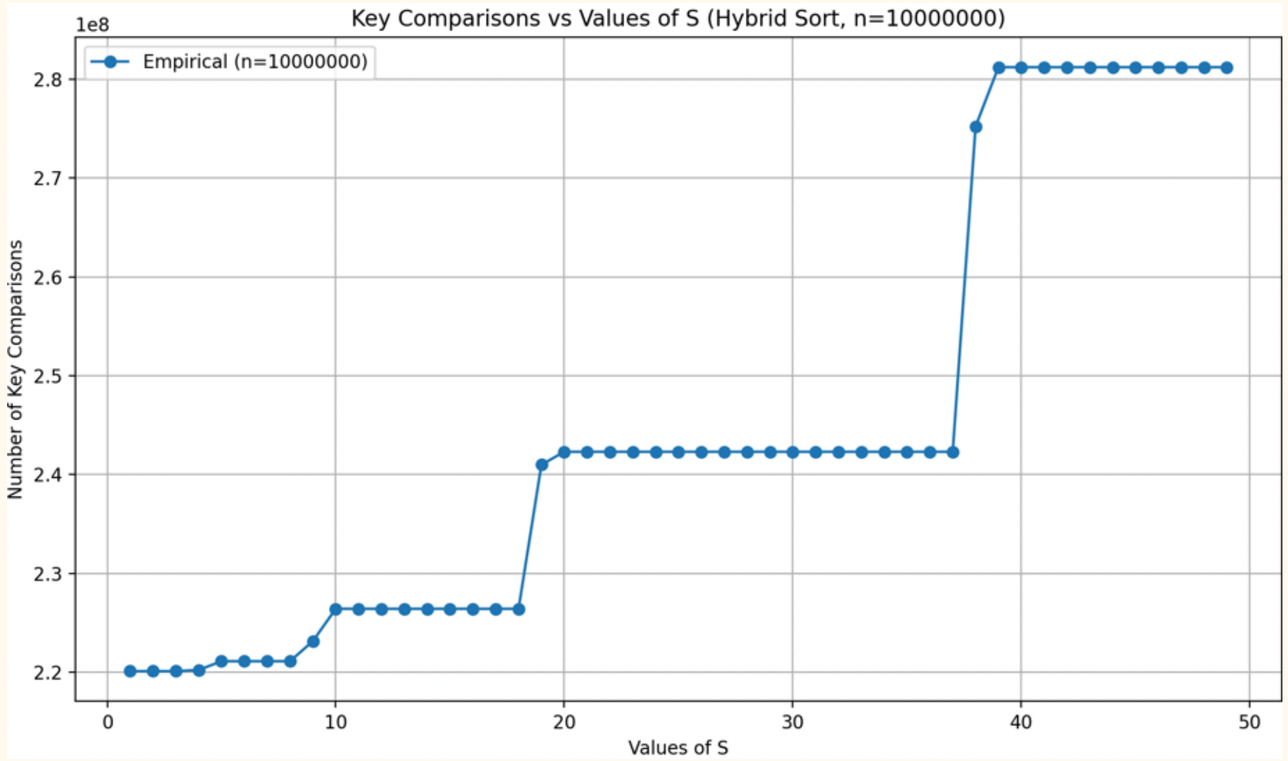


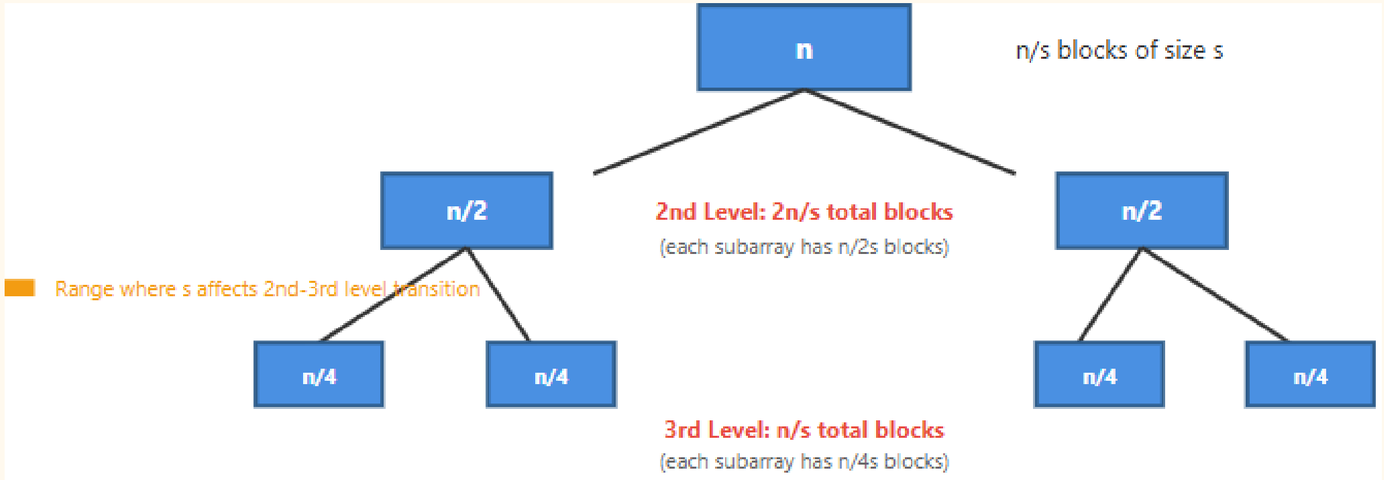


No. of key comparison is linearly correlated to input size (**n**)

Expected time complexity to be O(ns + n log n/s)

Increasing the value of **s** directly increases the number of comparisons, reducing efficiency.





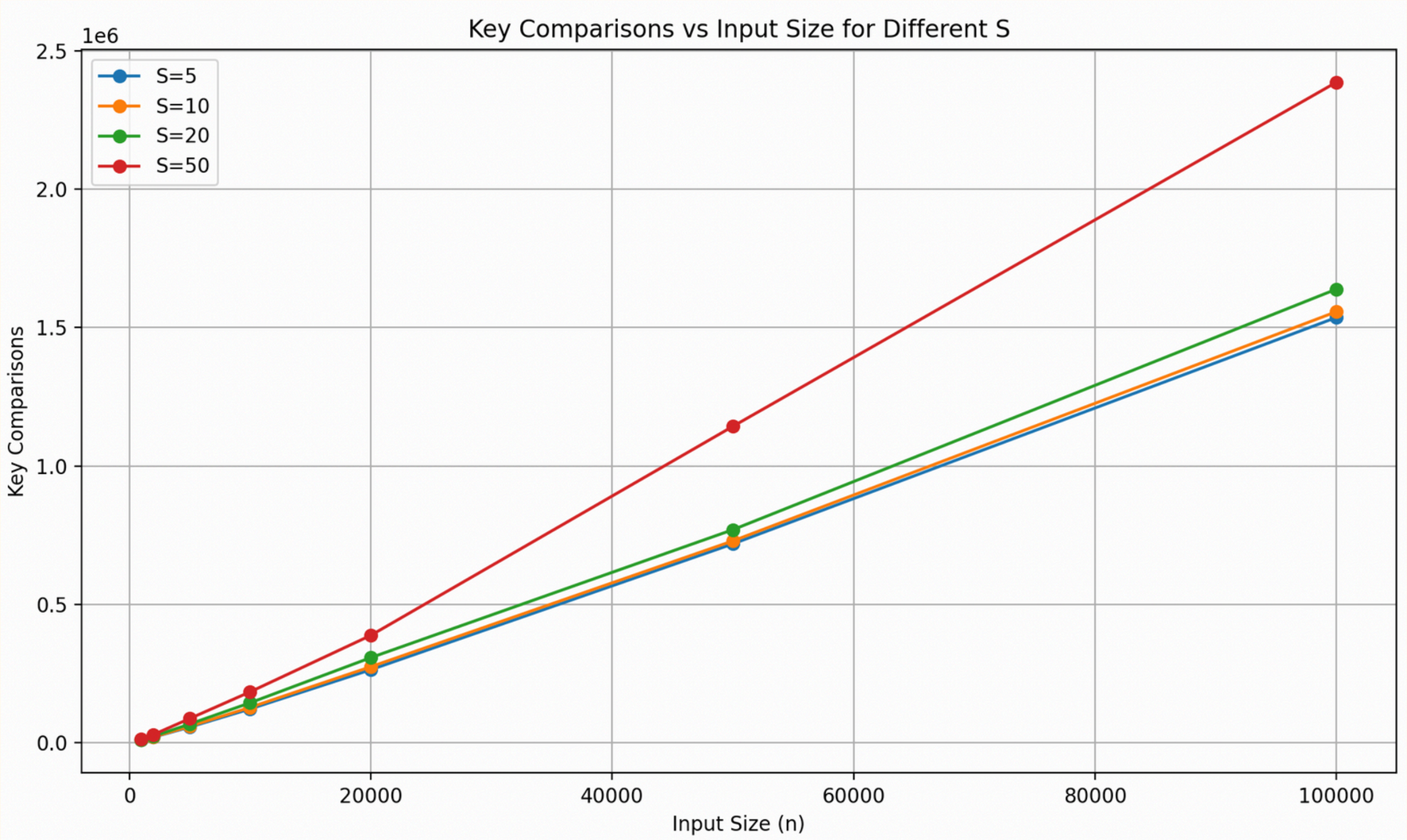
Based on the graph:

Staircase shape, why ?

- array is split in half until each subarray is of size ≤ **s**
- As **s** increases, the array is split into fewer, larger subarrays
- If **s** is between subarray sizes **2n/s** and **n/s**, Insertion Sort will only be invoked for subarrays of size **n/s**
- the flat regions occur when **s** is in a range that doesn't affect the recursion tree's depth
- The steps occur when **s** crosses a boundary
- Each step width doubles compared to the previous one

Fixed input value (n)

No. of Key comparison Vs Different (s) value



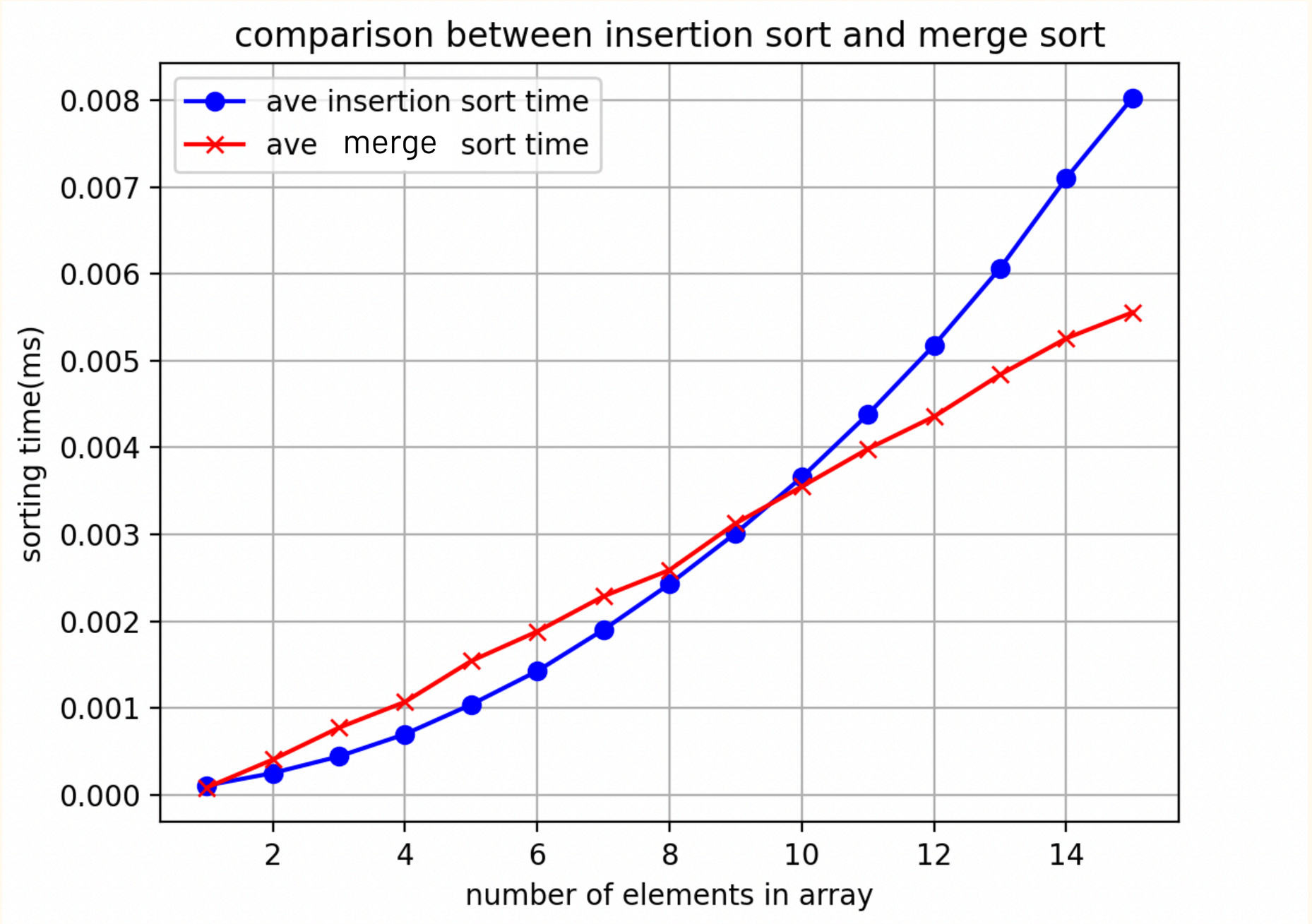
Finding(s)

Optimum value

larger s=>more comparisons

smaller s=>lesser comparisons


Time Taken vs Input Size for Different S
S=5
S=10
S=20
S=50
Time Taken (seconds)
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0
20000
40000
60000
80000
100000
Input Size (n)



- insertion sort is more effective than merge sort for smaller arrays

Using different s

Finding time taken for n= 10 million

```
def main():
    gc.collect()
    random.seed(0)

    n = 10000000
    input_data = [random.randint(1, n) for _ in range(n)]
    NUMBEROFRUNS = 10
    comparisons = []
    ave_time_taken = []

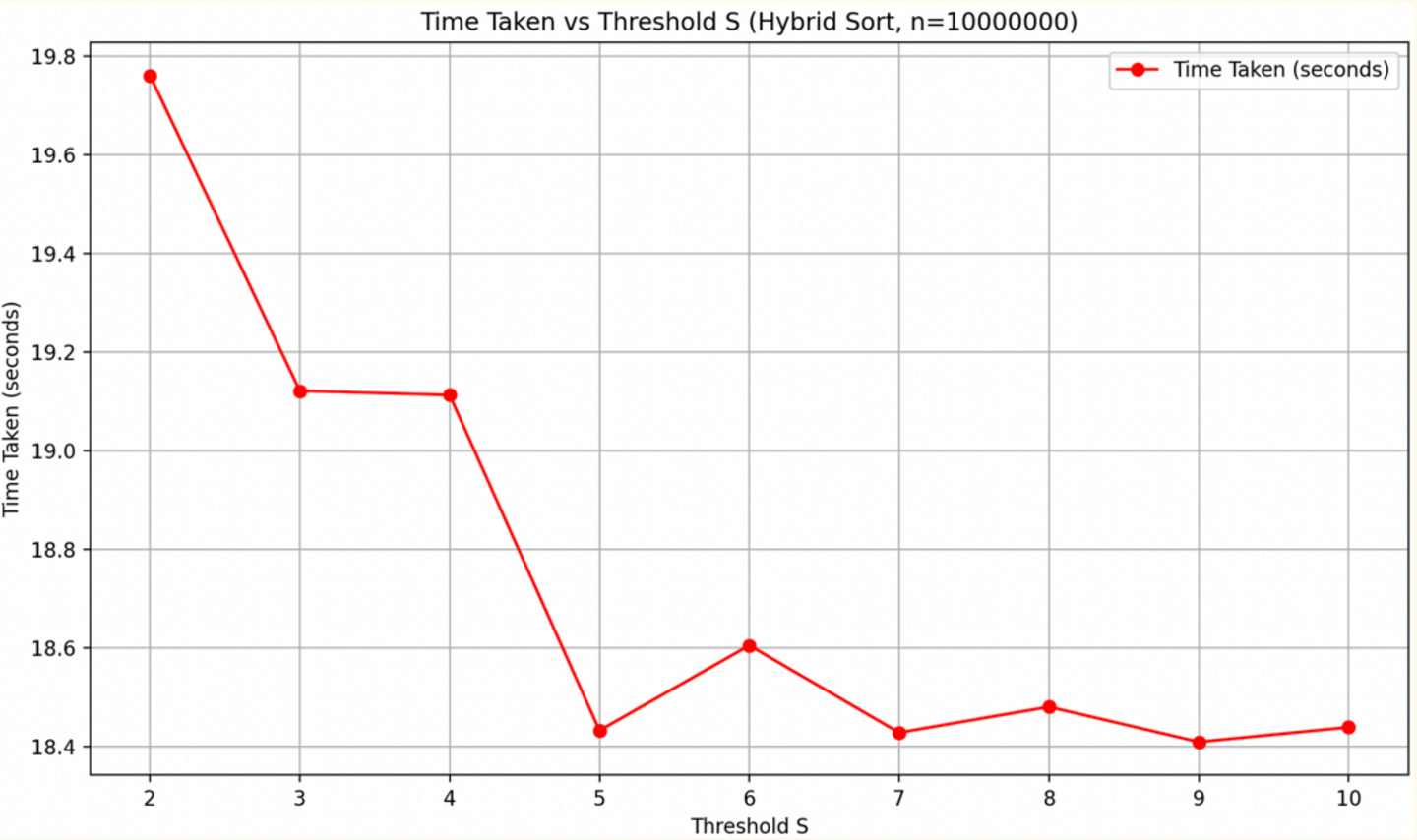
    for S in range(1, 11):
        # Measure comparisons
        comp_obj = KeyComparisons()
        data_copy = input_data.copy()
        hybrid_sort(data_copy, S, comp_obj)
        comparisons.append(comp_obj.key_comparisons)
```

```
total_time = 0.0
for _ in range(NUMBEROFRUNS):
    data_copy = input_data.copy()
    gc.collect()
    start_time = time.process_time()
    hybrid_sort(data_copy, S, KeyComparisons())
    end_time = time.process_time()
    total_time += (end_time - start_time)

avg_time = total_time / NUMBEROFRUNS
ave_time_taken.append(avg_time)
```

Finding(s)

Optimum value




```
10 runs
S: 2, Key Comparisons: 220098832, Average Time: 19.7614 seconds
S: 3, Key Comparisons: 220099977, Average Time: 19.1213 seconds
S: 4, Key Comparisons: 220222280, Average Time: 19.1128 seconds
S: 5, Key Comparisons: 221105852, Average Time: 18.4321 seconds
S: 6, Key Comparisons: 221105852, Average Time: 18.6047 seconds
S: 7, Key Comparisons: 221105852, Average Time: 18.4279 seconds
S: 8, Key Comparisons: 221105852, Average Time: 18.4801 seconds
S: 9, Key Comparisons: 223151201, Average Time: 18.4090 seconds
S: 10, Key Comparisons: 226424106, Average Time: 18.4387 seconds
```

S=5, ave time = 18.4321s

S=7, ave time = 18.4279s

S=9, ave time = 18.4090s

With S = 5,7,9

Running algorithm again

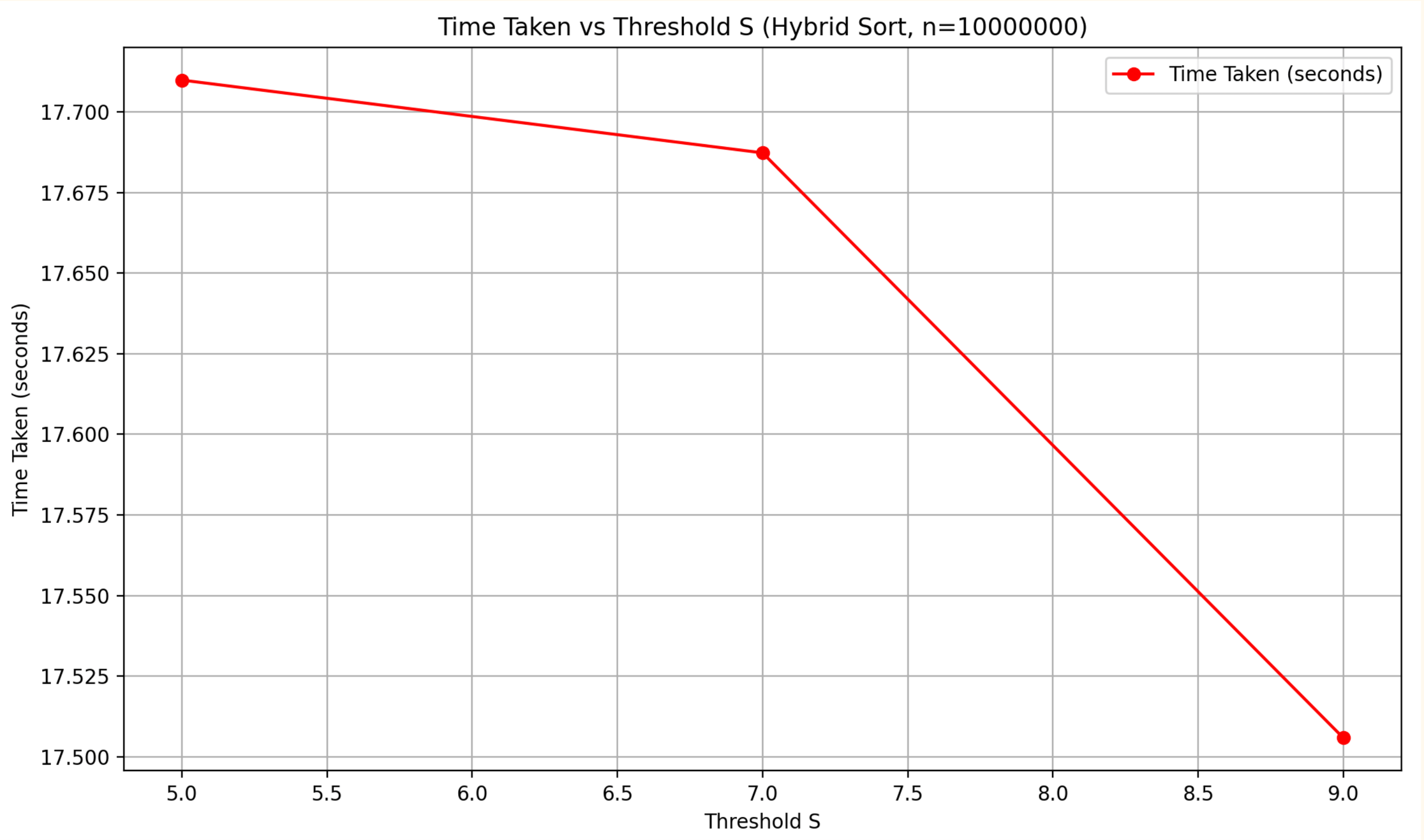
```python
def main():
    gc.collect()
    random.seed(0)

    n = 10000000
    input_data = [random.randint(1, n) for _ in range(n)]
    NUMBEROFRUNS = 30

    threshold_S = [5,7,9]
    comparisons = []
    ave_time_taken = []
```

```python
    #Measure average time over multiple runs
    total_time = 0.0
    for _ in range(NUMBEROFRUNS):
        data_copy = input_data.copy()
        gc.collect()
        start_time = time.process_time()
        hybrid_sort(data_copy, S, KeyComparisons())
        end_time = time.process_time()
        total_time += (end_time - start_time)

    avg_time = total_time / NUMBEROFRUNS
    ave_time_taken.append(avg_time)
```


Time Taken vs Threshold S (Hybrid Sort, n=10000000)
Time Taken (seconds)
17.700
17.675
17.650
17.625
17.600
17.575
17.550
17.525
17.500
Time Taken (seconds)
5.0
5.5
6.0
6.5
7.0
7.5
8.0
8.5
9.0
Threshold S

Conclusion :
Optimal value of S = 9 for n = 10 million

Comparing Merge Sort and Hybrid sort

```python
def main():
    random.seed(0)
    n = 10000000
    runs = 10
    S = 9
    input_data = [random.randint(1, n) for _ in range(n)]
```

```python
# MERGE SORT
total_time_merge = 0.0
for _ in range(runs):
    data_copy = input_data.copy()
    comp_obj = KeyComparisons()
    gc.collect()
    start = time.process_time()
    merge_sort(data_copy, comp_obj)
    end = time.process_time()
    total_time_merge += (end - start)
avg_time_merge = total_time_merge / runs

print(f"Merge Sort")
print(f"Key Comparisons: {comp_obj.key_comparisons}")
print(f"Avg CPU Time: {avg_time_merge:.4f} seconds\n")
```

```python
# HYBRID SORT
total_time_hybrid = 0.0
for _ in range(runs):
    data_copy = input_data.copy()
    comp_obj = KeyComparisons()
    gc.collect()
    start = time.process_time()
    hybrid_sort(data_copy, S, comp_obj)
    end = time.process_time()
    total_time_hybrid += (end - start)
avg_time_hybrid = total_time_hybrid / runs

print(f"Hybrid Sort (S = {S})")
print(f"Key Comparisons: {comp_obj.key_comparisons}")
print(f"Avg CPU Time: {avg_time_hybrid:.4f} seconds")
```

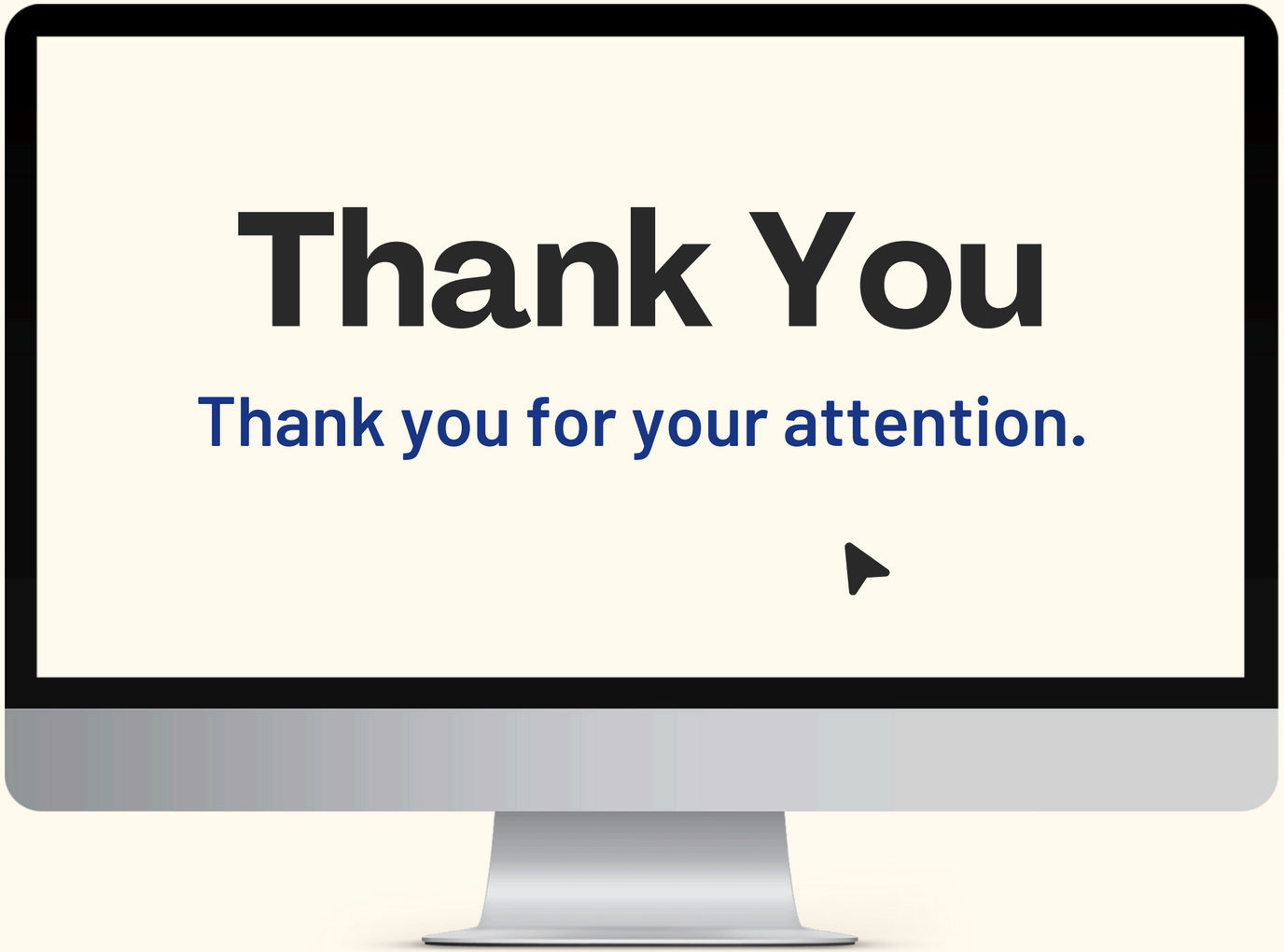
```
Merge Sort
Key Comparisons: 220098832
Avg CPU Time: 19.1034 seconds

Hybrid Sort (S = 9)
Key Comparisons: 223151201
Avg CPU Time: 17.6852 seconds
```

Comparison

Merge vs Hybrid(with optimal S=9)

Hybrid sort was faster than merge sort by 1.42 seconds


Thank You
Thank you for your attention.